UNIVERSITY OF COLOMBO, SRI LANKA

UNIVERSITY OF COLOMBO SCHOOL OF COMPUTING

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

First Year Examination – Semester I –2022

SCS-1201 – Data Structures and Algorithms I

TWO (2) HOURS

261

Answer ALL Questions

To be completed by the candidate

Examination Index No: .....

Important Instructions to candidates:

1. Students should answer in the medium of **English language only** using the space provided in this question paper.
2. Note that questions appear on both sides of the paper. If a page or a part of this question paper is not printed, please inform the supervisor immediately.
3. Write your index number **CLEARLY** on each and every page of this Question paper.
4. This paper consists of 4 questions in 12 pages (including the Cover Page).
5. Programmable Calculators or any electronic device capable of storing and retrieving text including electronic dictionaries, smart watches and mobile phones are **not allowed**
6. **Non-Programmable** calculators are allowed.
7. Do not tear off any part of this answer book. Under no circumstances may this book, used or unused, be removed from the Examination Hall by a candidate
8. The paper will total 100 marks (***25 for questions 1 and 4, 30 marks for question 2 and 20 marks for question 3***).

For Examiner's use only	
Question No	**Marks**
1	
2	
3	
4	
Total	

Index No: ...

Question 1

(a) Consider the following infix expression.

3+4*(5*3-6)

(i) Write the resultant postfix expression of the above if one uses the usual Stacks algorithm to convert the infix to postfix.

(4 Marks)

(ii) When one evaluates the postfix expression obtained from a(i), what is the **maximum number** of tokens that will appear on the stack **at a time** during the evaluation of the expression?

(4 Marks)

(iii) What is/are the stack's remaining content(s) after evaluating the expression?

(4 Marks)

(iv) Consider that a multiplication operator (*) is added to the end of the original expression. When evaluating the expression after converting it into postfix, what would be the last stack content and the buffer input token value?

(5 Marks)

(b) (i) If one evaluates the following pseudocode algorithm segment using a dry run (hand execute), what would be the final content of the queue?

```
q = queue();
for(j=1; j<5;j++)
{
    q.enqueue (j);
}
for(i=1;i<5;i++)
{
    if(i%2 == 0)
    {
    q.dequeue()
    }
    else{
    q.enqueue(i*4)
    }
}
```

(4 Marks)

(ii) Hand execute (dry run) the following pseudocode segment and write the final content of the resulting stack.

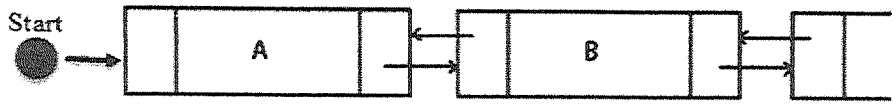
(4 Marks)

```
stack=[]
for  i = 1 to 12
  if i%4 <> 0 then
   stack.append( i )
  else
    if i%2=0 then
      stack.pop()
    endif
  endif
next i
```

Index No: ...

Question 2

(a) Consider the following straight doubly linked list with the initial pointer "***Start***".

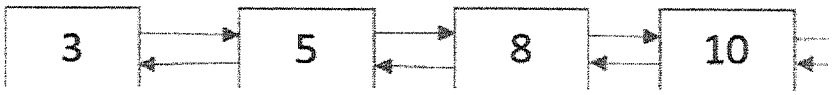


(i) Write a pseudocode or C code algorithm to remove Node B from the above List.

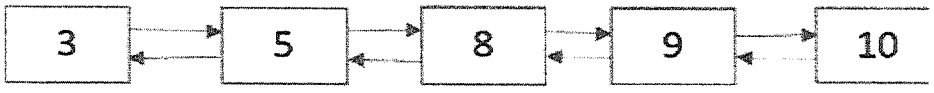
Note: (I) You may assume forward and backward reference as **next** and **prev** respectively.
(II) Do not use additional pointers other than the given pointes.

(2 Marks)

(ii) Write a pseudo code algorithm to insert a new node into a sorted linked list whilst maintaining the sorted order. An example test case is given below.
As a test case, when you test the algorithm, you may use the content of the new node as nine (09) and the initial list as given below.



After inserting the new node (09), the final sorted doubly linked list is as follows:



(4 Marks)

(iii) A polynomial p(x) is the expression in variable x which is in the form of $ax^n + bx^{n-1} + ... + jx + k$, where a, b, c ...., j, k fall in the category of real numbers and 'n' is a non-negative integer

Describe, how to represent the above polynomial using a singly linked list.

(2 Marks)

(iv) Consider the following two polynomials:

Polynormal 1 :$5x^2 + 4x^1 + 2x^0$
Polynormal 2 :$5x^1 - 5x^0$

Using the features of the linked lists, write a pseudocode algorithm to subtract polynomial 2 from polynomial 1.

(4 Marks)

(b) The Fibonacci numbers are a sequence of integers in which the first two elements are 0 & 1, and each following element is the sum of the two preceding elements:

(i) Write a formula to generate the n^{th} Fibonacci number.

(2 Marks)

(ii) Write a recursive pseudocode algorithm to find the n^{th} Fibonacci number

(3 Marks)

(iii) Prove the time complexity of the recursive Fibonacci algorithm as O(2^n)).

(4 Marks)

(iv) Consider the following relationship related to the running time of the algorithm.

$T(n)=4n^3+n^2+2n+5$

Prove the complexity of the algorithm as $O(n^3)$ using Big-Oh Rules.

(3 Marks)

(v) Which parameters best describe the criteria for comparing the efficiency of algorithms? Justify the answer.

(3 Marks)

(vi) Consider following time complexity of algorithms.as any input of size n:

$T(n) =n^2+3n+5$

"When estimating running time of a relation for larger n values, there is not significant different when considering all the terms in the relation or considering the most dominant component of the relation"

Prove the validity of the above expression by substituting different values for n in the above relationship.

(3 Marks)

Question 3

(a) One wants to create a max-heap using the following set of integers.

[6,2,14,1,7,21]

Nodes are inserted one by one and any heap property violation is fixed when detected.

Which of the following are valid/invalid statements?

(i) During the max heap creation, 14 is swapped with 6
(ii) The total number of swaps during the heap creation is 5.
(iii) When creating the max heap, 21 is swapped with 6, and again 21 is swapped with 14.
(iv) At the intermediate stage of heap construction, 14 is a root value.
(v) 2 never becomes a root value during the heap construction

(5 Marks)

(i).....
(ii).....
(iii).....
(iv).....
(v).....

(b) Consider the following two techniques when selecting a pivot value for partitioning a file in a quick sort algorithm.

- Pick the first element as the pivot.
- Pick median as the pivot.

Compare the suitability of the above techniques when one applies the partitioning algorithm for the following cases.

(i) All the elements in the file are already sorted in increasing order.
(ii) The elements in the file are arranged in random order.

(5 Marks)

(i)	(ii)

(c)Suppose one sorts an array of ten integers using quicksort, and if he/she has just finished the first partitioning as follows:

16,23,40,57,89,90,75,92,67,100

What are all the possible pivot values that can choose to obtain the above intermediate output?

(5 Marks)

(d) Consider the following straight merge sort algorithm

Step 1 : Divide the file into n sub files of size 1

Step 2: merge adjacent pairs of files according to their values in increasing order (Then we have approximately n/2 files of size 2)

Step 3: Repeat this process (step 1 and step 2) until there is only one file remaining of size n.

If one uses the above straight sort algorithm to sort the following data set, how many passes(rounds) are required to completely sort the data set

Data set is: 25,57,48,37,12,92,86,33

(5 Marks)

Index No: ...

Question 4

(a) (i) The output of the level order traversal is given below:

7, 4, 12, 3, 6, 8, 1, 5, 10

Consider the following algorithm

Step 1: First pick the first element of the array and make it root.
Step 2: Pick the second element, if its value is smaller than root node value makes it left child,
Step 3: Else make it right child

The recursive call Step 2 and Step 3 continued to make the tree until all the nodes are picked.

What would be the resultant tree if one applies the above data set to the algorithm?

(4 Marks)

(ii) What is the minimum and the maximum number of nodes in an AVL tree of height 3 and 4 respectively?

Hint: The height of a tree consisting of single node is 0, Give an exact number for the answer. Do not provide the formula.

(4 Marks)

	Height=3	Height=4
Max. number of Nodes		
Min. Number of Nodes		

(iii)What is the Maximum and the minimum height of an AVL tree when the total number of nodes is equal to 8, 10 and 13 respectively

(4 Marks)

No. of Nodes	Maximum Height	Minimum Height
8		
10		
13		

(b) (i) Draw the weighted directed graph represented by the adjacency matrix below. A nonzero value at [row, column] indicates that the vertex in the row is adjacent to the vertex in the column:

(3 Marks)

	P	Q	R	S	T
P	1	0	0	1	1
Q	1	0	1	0	0
R	1	0	0	0	1
S	1	1	0	0	0
T	1	0	0	1	0

(ii) Describe how one can find the path matrix using the given adjacency matrix.

(4 Marks)

Index No: ...

(iii) Write the path matrix according to the methodology described in part b(ii)) above

(3 Marks)

(iv) Describe the Breadth-First Traversal of a graph proposed by you in part b(i) above using a queue.

(3 Marks)

****

UNIVERSITY OF COLOMBO, SRI LANKA

UNIVERSITY OF COLOMBO SCHOOL OF COMPUTING

Bachelor of Science in Computer Science

First Year Examination - Semester 1 - 2022

SCS 1202 — Programming using C

057

TWO HOURS

To be completed by the candidate								
Index Number								

Important Instructions

- This paper having **12 pages** is in two parts, **Part A** and **Part B**. Part A consists of 20 MCQs and Part B consists of 5 questions. **The candidate must answer all questions.**
- **Write your answers only in the answer boxes provided** on this question paper.
- Do not tear off any part of this answer book. Under no circumstances may this book (or any part of this book), used or unused, be removed from the examination hall by a candidate.
- Questions appear on both sides of the paper. If a page is not printed, please inform the supervisor immediately.
- Any electronic device capable of storing and retrieving text including electronic dictionaries and mobile phones are **not allowed**.
- Calculators are **not allowed**.
- Please write your answers only in English.
- The C operator precedence table is given on page 12.

To be completed by the examiners

A	
1	
2	
3	
4	
5	
Total	

Part A

In each of the questions **1** *to* **15**, *pick one of the alternatives from (1), (2), (3) and (4) which is* **correct or most appropriate** *and write your response in the answer box provided.*

[50 marks]

1. If *fahr=98* what will be the output of the following C fragment?

```
celsius = 5 * (fahr - 32) / 9;
printf("%d\t%d\n", fahr, celsius);
```

(1) 9836 (2) 98 36 (3) 98\t36 (4) 98 0

ANSWER:

2. If *fahr=98* what will be the output of the following C fragment?

```
celsius = 5/9 * (fahr - 32);
printf("%d\t%d\n", fahr, celsius);
```

(1) 9836 (2) 98 36 (3) 98\t36 (4) 98 0

ANSWER:

3. Which of the following codes show proper indentation and spacing?

(1)

```
#include <stdio.h>
int main() {
int c;
for (c = 1; c <= 5; c++)
printf("Hello\n");
}
```

(2)

```
#include <stdio.h>

int main() {
  int c;

  for (c = 1; c <= 5; c++)
  printf("Hello\n");
}
```

(3)

```
#include <stdio.h>

int main() {
  int c;

  for (c = 1; c <= 5; c++)
        printf("Hello\n");
    }
```

(4)

```
#include <stdio.h>

int main() {
  int c;

  for (c = 1; c <= 5; c++)
    printf("Hello\n");
}
```

ANSWER:

4. Which of the following is **not** a standard C datatype?

(1) char (2) double (3) int (4) real

ANSWER:

5. If i is 2 and j is 10, then what will be the value of i at the end of the following *while* loop?

```
while (i<j)
  i = 2*i;
```

(1) 2 (2) 4 (3) 8 (4) 16

ANSWER: ☐

6. What will be the result if the following C program is compiled and (if possible) executed?

```
#include <stdio.h>

int main() {
  char  a;
  a = 'A';
  printf("a=%c %d\n",a,a);
}
```

(1) compilation error (2) a=A 65 (3) a=a a (4) a=a 65

ANSWER: ☐

7. What will be the result if the following C program is compiled and executed?

```
#include <stdio.h>

int main() {
  int x=12,y=3;
  printf("Ans1=%d Ans2=%d\n",x/y,x%y);
}
```

(1) Ans1=0 Ans2=4 (2) Ans1=0.0 Ans2=4.0 (3) Ans1=4 Ans2=0 (4) Ans1=4.0 Ans2=0.0

ANSWER: ☐

8. If n, x and y are integers and n is 6, then the C statements x=n++; y=++x; would result in

(1) n=6, x=6 and y=7. (2) n=6, x=7 and y=7.
(3) n=7, x=6 and y=7. (4) n=7, x=7 and y=7.

ANSWER: ☐

9. Consider the following statement that is written in a C code:

```
#define func(X,Y)   ((X)>(Y) ? (X):(Y))
```

Assume that in the same code, the following statements are written:

```
x = 19;
y = 20;
z = func(x,y);
```

If the above statements are executed, what will be placed in z?

(1) ? (2) : (3) 19 (4) 20

ANSWER: ☐

10. Which of the following statements are true about the scope of C variables?

A- Variables defined at the beginning of a program before all functions are visible to the entire program.

B- Variables defined inside functions are visible only to that function.

C- `extern int sp;` indicates that the integer variable *sp* is defined in another file.

(1) A and B only (2) A and C only (3) B and C only (4) All A, B and C ANSWER: ☐

11. A structure is defined as follows:

```
struct point {
  int x;
  int y;
} p1,p2;
```

Which of the following are valid?

A- `p1.x = 5; p1.y = 6;` are valid statements.

B- `p2` is a variable of type `struct point`.

C- The values of the x and y variables are fixed and cannot be changed.

(1) A and B only (2) A and C only (3) B and C only (4) All A, B and C ANSWER: ☐

12. Which of the following are correct?

A- A character written between single quotes represents an integer value equal to the numerical value of the character in the machine's character set.

B- `char myMessage[] = "Error!";` Here myMessage is an array, just big enough to hold the sequence of characters :*Error!* and `'\0'` that ends it.

C- `if (S[i] >= '0' && S[i] <= '9')`

checks whether the character in S[i] is a digit.

(1) A and B only (2) A and C only (3) B and C only (4) All A, B and C ANSWER: ☐

13. What will be the output of the following C code?

```
#include <stdio.h>

int main(){
  printf("Answer=%d\n",(2*10) + 20 * 30);
}
```

(1) Answer=620 (2) Answer=1200 (3) Answer=1210 (4) Answer=1800 ANSWER: ☐

14. What will be the output of the following C code?

```
#include <stdio.h>

int main(){
  printf("Answer=%d\n",100/10*10%3);
}
```

(1) Answer=0 (2) Answer=1 (3) Answer=10 (4) Answer=100

ANSWER: [ ]

15. What will be the output of the following C code?

```
int main(){
  short I=0xB;
  short J=0xC;
  short N;
  N = I|J;
  printf("N=%x\t",N);
  printf("N=%d\n",N);
}
```

(1) N=7 N=7 (2) N=8 N=8 (3) N=c N=12 (4) N=f N=15

ANSWER: [ ]

16. How many times will **Hello** be printed if the following loop is executed?

```
int i,j;

for (i=0; i<3; i++)
  for (j=0; j<2; j++)
    printf("Hello\n");
```

(1) 3 (2) 6 (3) 9 (4) 12

ANSWER: [ ]

17. Which of the following are correct?

A-The return value of main() could be passed to the operating system to show how the program exited.

B-The functions that we write could be placed in a library to be used by other people.

C-One cannot read *stdio.h* file as it is in binary.

(1) A and B only (2) A and C only (3) B and C only (4) All A, B and C

ANSWER: [ ]

18. Consider the following code:

```
#include <stdio.h>

int main() {
   int i;
   double number, sum = 0.0;

   for (i = 1; i <= 6; ++i) {
      printf("Enter n%d: ", i);
      scanf("%lf", &number);

      if (number < 0.0)
         continue;

      if (number == 0.0)
         break;

      sum += number;
   }

   printf("Sum = %.2lf", sum);
   return 0;
}
```

If the numbers $7, 4, 9, -1$ and 0 are input to the above code, then which of the following will be the output?

(1) 7 (2) 11 (3) 19 (4) 20

ANSWER: [ ]

19. What will be the values of y and z[0] at the end of execution of the following C fragment?

```
int x=50, y=10, z[10];
int *ip;

ip = &x;
y = *ip;
*ip = 20;
z[0] = *ip;
ip = &z[0];
*ip = *ip + 10;
y = *ip;
```

(1) y=address of ip, z[0]=30 (2) y=address of ip, z[0]=50 (3) y=30, z[0]=30 (4) y=50, z[0]=50

ANSWER: [ ]

20. If a is an array and i is an integer, then which of the following are equivalent?

(1) `a[i],a+i` (2) `a[i],*(a+i)` (3) `a[i], *a+i` (4) `&a[i],a+i`

ANSWER: [ ]

Part B

1. Assume the following extract, named *average.c* is taken from a *correct* C code and thus no compilation errors occur. Also assume that . . . contain valid C content.

```
#include <stdio.h>

#define N 100

int main() {
    ...
    for (i=0; i<N; i++)
...

#ifdef PRINT
    printf(...);
#endif
}
```

Explain the sequence of steps that takes place when the above code is compiled using:
cc -o average average.c -DPRINT

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

[10 marks]

2. (a) The following incomplete code named *argCheck.c* is designed to print its command line arguments when it is compiled and executed.

Fill the **two** blanks in it (indicated by --------) to complete it. **Note:** Assume the program is compiled using `gcc -o argCheck argCheck.c`.

```
#include <stdio.h>
int main(int argc,char *argv[]) {
  int i;
  for (i=0;i< --------;i++)   /* blank 1 */

    printf("%s ",----------------); /* blank 2 */
  printf("\n");
  return 0;
}
```

[3 marks]

(b) The following incomplete function is designed to return the length of a string *s*.

Fill the **three** blanks in it (indicated by --------) to complete it.

```
/* strlen: return length of string s */
int strlen(char *s) {
  int n;
  for (n=0; *s != --------; --------) /* blanks 1 and 2 */
    n++;
  return --------;   /*blank 3 */
}
```

[4 marks]

(c) The following incomplete function is designed to copy string *t* to string *s*.

Fill the **three** blanks in it (indicated by --------) to complete it.

```
/* strcpy: copy t to s; pointer version */
void strcpy(char *s, char *t)  {

  while ((*s = --------) != --------) {  /* blanks 1 and 2 */
    s++;

    --------; /* blank 3 */
  }
}
```

[3 marks]

3. The following incomplete code is designed to read numbers from a file into an array, print the numbers in the array and its maximum.

Fill the **five** blanks in it (indicated by --------) to complete it.

```
#include <stdio.h>

int main() {
  int numbers[50], maximum;

  int i = 0;
  FILE *file;

  if (file = --------("numberFile.txt", "r")) { /* blank 1 */

    while (--------(file, "%d", &numbers[i]) != EOF)  { /* blank 2 */
      i++;
    }

    --------(file); /* blank 3 */

    numbers[i] = '\0';

    printf("The input array is:\n");
    for (i = 0; numbers[i] != '\0'; i++)
      printf("%d\t", numbers[i]);
    printf("\n");

    maximum = ----------------;  /* blank 4 */

    for (i = 1; numbers[i] != '\0'; i++)
      if (numbers[i] > maximum)

          ------------------------;  /* blank 5 */

    printf("Maximum element in array is %d.\n", maximum);
    return 0;
  }
}
```

[10 marks]

4. A program written in C is needed to multiply two *integer* $N \times N$ matrices A and B to get the result matrix C.

Note: In matrix multiplication, to compute the element $C[i][j]$, the elements in the i^{th} row of matrix A should be multiplied with the relevant elements in the j^{th} column of matrix B. E.g., In a 3×3 matrix multiplication:

$c[0][0] = a[0][0] \times b[0][0] + a[0][1] \times b[1][0] + a[0][2] \times b[2][0]$

Figure 1 is an example of a 2×2 matrix multiplication:

$$AB = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

Figure 1: Matrix - matrix multiplication example

An *incomplete* program made for the purpose is given below. It is to run in a continuous loop until EOF (Ctrl+D) is pressed. At each iteration, the A and B matrices are input interactively by the user. After the computation, the result matrix C should be displayed on the screen in matrix format with the elements displayed row-wise and with a space between the row elements.

Complete the code by **filling-in** the 10 blanks indicated by dashes (--------).

[10 marks]

```
/* mm.c -  Matrix Multiplication */

#include <stdio.h>
#include <stdlib.h>

#define N    4    /* Matrix Size */

void matmult(int inA[N][N], int inB[N][N], int outC[N][N]);

void matmult(--------, --------, --------) {/* blanks 1,2,3 */
    int i, j, k;

    ---------------------------- { /* blank 4 */

        -------------------------------{ /* blank 5 */

            ----------;  /* blank 6 */
            for (k=0; k<N; k++)

                ------------------------------------------; /* blank 7 */
        }
    }
}
```

```
int main(int argc, char *argv[]) {
    int i, j, k, choice;
    int A[N][N];
    int B[N][N];
    int C[N][N];

    do {
        /* input matrix A  */
        for (i=0; i<N; i++)
            for (j=0; j<N; j++) {
                printf("Enter A[%d][%d]\n",i,j);
                scanf("%d",&A[i][j]);
            }

        /* input matrix B  */
        for (i=0; i<N; i++)
            for (j=0; j<N; j++) {
                printf("Enter B[%d][%d]\n",i,j);
                scanf("%d",&B[i][j]);
            }

        /* do matrix multiplication */

        ------------------; /* blank 8 */

#ifdef PRINT
        printf("Answer c:\n");
        for (i=0; i<N; i++){
            for (j=0; j<N; j++)

                --------------------------------; /* blank 9 */

            -----------; /* blank 10 */
        }
#endif
        printf("Press ENTER key to continue and ^D to exit!\n");
        choice = getchar();
        choice = getchar();
    } while ( choice != EOF );

  return(0);
}
```

5. The following incomplete code is designed to use recursion to print the sum of first n natural numbers using recursion. Fill the **five** blanks in it (indicated by --------) to complete it.

Note: For example, if n is input as 5, then the program should output 15 (i.e., 1+2+3+4+5).

```
#include <stdio.h>

int sum(int n);

int main() {
    int number, result;

    printf("Enter a positive integer: ");
    scanf("%d", &number);

    --------------------------------; /* blank 1 */

    printf("sum = %d\n", result);
    return 0;
}

int ------------------------{ /* blank 2 */

    ---------------- /* blank 3 */
        /* function calls itself */

        --------------------------------;  /* blank 4 */
    else

        return --------; /* blank 5 */
}
```

[10 marks]

Operator	Associativity
( ) [] -> .	left to right
! ~ ++ -- + - * & (type) sizeof	right to left
* / %	left to right
+ -	left to right
<< >>	left to right
< <= > >=	left to right
== !=	left to right
&	left to right
^	left to right
|	left to right
&&	left to right
||	left to right
?:	right to left
= += -= *= /= %= &= ^= |= <<= >>=	right to left
,	left to right

Figure 2: C operator precedence table